Министерство образования Республики Беларусь
Учреждение образования
«Брестский Государственный технический университет»
Кафедра ИИТ

Лабораторная работа №3
По дисциплине «Криптография и сетевая безопасность»
Тема: **«Блочные шифры и классические режимы шифрования»**

Выполнил:
Студент 4 курса
Группы АС-65
Ярмак К. А.
Проверил:
Хацкевич А. С.

Брест 2026

Цель: программная реализация симметричного блочного шифра (AES-128 или СТБ 34.101.31 «БелТ») и классических режимов работы (ECB, CBC, CFB, OFB, CTR) с поддержкой алгоритма дополнения блоков (Padding).

Вариант 12

В лабораторной работе требуется реализовать указанный блочный шифр и продемонстрировать его работу в двух режимах шифрования: обязательном базовом режиме ECB и дополнительном режиме согласно варианту.

Вариант	Базовый шифр	Режимы работы	Особенности реализации / Фокус тестирования
12	СТБ 34.101.31	ECB + OFB	Оптимизация предварительной генерации псевдослучайной гаммы для OFB.

1.1 Шифр СТБ 34.101.31 («БелТ»)

БелТ шифрует блок 128 бит ключом 256 бит за 8 раундов. Структура — модифицированная сеть Фейстеля. Используются три операции над 32-битными словами: XOR (⊕), сложение по модулю 2^32 (⊞) и вычитание по модулю 2^32 (⊟).

Данные и ключ
Блок делится на четыре слова a, b, c, d. Ключ делится на восемь слов K1…K8. Раундовых подключей 56: в каждом раунде подключ прибавляется в семи шагах из девяти, поэтому 8 × 7 = 56. Подключи получаются циклическим повторением слов ключа.

H-блок
Фиксированная таблица подстановок H из 256 байт, она даёт нелинейность. Функция G_r применяет H к каждому из четырёх байт слова u и циклически сдвигает результат влево на r бит (r ∈ {5, 13, 21}).

Раунд i = 1…8 (используются k[7i−6] … k[7i]):

1. b ← b ⊕ G5(a ⊞ k[7i−6])
2. c ← c ⊕ G21(d ⊞ k[7i−5])
3. a ← a ⊟ G13(b ⊞ k[7i−4])
4. e ← G21(b ⊞ c ⊞ k[7i−3]) ⊕ ⟨i⟩32
5. b ← b ⊞ e
6. c ← c ⊟ e
7. d ← d ⊞ G13(c ⊞ k[7i−2])
8. b ← b ⊕ G21(a ⊞ k[7i−1])
9. c ← c ⊕ G5(d ⊞ k[7i])

После шага 9 слова переставляются: a ↔ b, c ↔ d, b ↔ c. После 8 раундов шифротекст собирается как Y = b ‖ d ‖ a ‖ c. Расшифрование выполняет те же преобразования в обратном порядке с обратными операциями и подключами с конца.

Режим ECB

Каждый блок открытого текста шифруется независимо от остальных:
C_i = E_K(P_i), P_i = D_K(C_i)

Одинаковые блоки открытого текста дают одинаковые блоки шифротекста, поэтому в шифротексте видны статистические закономерности. Режим требует длины, кратной 16 байтам, то есть padding.

Режим OFB

Блочный шифр используется как генератор гаммы, которая не зависит ни от открытого текста, ни от шифротекста:
I_0 = IV, I_i = E_K(I_{i-1}), C_i = P_i ⊕ I_i, P_i = C_i ⊕ I_i

Зашифрование и расшифрование — одна и та же операция. Используется только E_K, функция D_K не нужна. Padding не требуется: последний блок гаммы обрезается до длины сообщения. Из независимости гаммы от данных следует, что её можно сгенерировать заранее; это и есть фокус варианта 12. Для разных сообщений с одним ключом IV должен быть разным: при повторе IV гаммы совпадают, и XOR двух шифротекстов равен XOR открытых текстов.

belt.py — функция G, раундовые ключи и зашифрование блока

```
def _G(u, r):
    t = (H[u & 255] | H[(u >> 8) & 255] << 8 | H[(u >> 16) & 255] << 16 | H[(u >> 24) & 255]
<< 24)

    return ((t << r) | (t >> (32 - r))) & M32


@lru_cache(maxsize=16)
def _expand(key):
    if len(key) != 32:
        raise ValueError("ключ БелТ должен быть 32 байта")

    w = struct.unpack("<8I", key)

    return tuple(w[i % 8] for i in range(56))


def encrypt_block(pt_block, key):
    if len(pt_block) != 16:
        raise ValueError("блок должен быть 16 байт")

    ks = _expand(bytes(key))
    a, b, c, d = struct.unpack("<4I", pt_block)
```

```
    for i in range(1, 9):
        k1, k2, k3, k4, k5, k6, k7 = ks[7 * i - 7:7 * i]

        b ^= _G((a + k1) & M32, 5)                  # 1
        c ^= _G((d + k2) & M32, 21)                 # 2
        a = (a - _G((b + k3) & M32, 13)) & M32      # 3
        e = _G((b + c + k4) & M32, 21) ^ i          # 4
        b = (b + e) & M32                           # 5
        c = (c - e) & M32                           # 6
        d = (d + _G((c + k5) & M32, 13)) & M32      # 7
        b ^= _G((a + k6) & M32, 21)                 # 8
        c ^= _G((d + k7) & M32, 5)                  # 9

        a, b = b, a                                 #  a ⇔ b
        c, d = d, c                                 #  c ⇔ d
        b, c = c, b                                 #  b ⇔ c

    return struct.pack("<4I", b, d, a, c)
```

modes.py — padding PKCS#7

```
def pad(data):
    k = B - len(data) % B

    return data + bytes([k]) * k


def unpad(data):
    if not data or len(data) % B:
        raise ValueError("неверная длина данных")

    k = data[-1]

    if not 1 ⩽ k ⩽ B or data[-k:] ≠ bytes([k]) * k:
        raise ValueError("неверный padding")

    return data[:-k]
```

modes.py — режим ECB

```
def ecb_encrypt(data, key):
    data = pad(data)

    return b"".join(encrypt_block(data[i:i + B], key) for i in range(0, len(data), B))


def ecb_decrypt(data, key):
    if len(data) % B:
        raise ValueError("длина шифротекста не кратна 16")

    pt = b"".join(decrypt_block(data[i:i + B], key) for i in range(0, len(data), B))

    return unpad(pt)
```

modes.py — режим OFB с предварительной генерацией гаммы

```
def ofb_gamma(key, iv, length):
    out = bytearray()
    blk = iv
```

```
    for _ in range(-(-length // B)):
        blk = encrypt_block(blk, key)
        out += blk

    return bytes(out[:length])


def ofb_crypt(data, key, iv, gamma=None):
    n = len(data)

    if gamma is None:
        gamma = ofb_gamma(key, iv, n)

    if len(gamma) < n:
        raise ValueError("гаммы меньше чем данных")

    x = int.from_bytes(data, "big") ^ int.from_bytes(gamma[:n], "big")

    return x.to_bytes(n, "big")
```

```
ECB/OFB round-trip: OK
len= 5 -> padded=16, ECB ct=16, OFB ct=5
len=16 -> padded=32, ECB ct=32, OFB ct=16
len=21 -> padded=32, ECB ct=32, OFB ct=21
ECB: уникальных блоков 2 из 9
OFB: уникальных блоков 8 из 8
naive: 0.584s | гамма заранее: 0.561s | XOR с готовой гаммой: 0.00068s
```

Вывод: программно реализовали симметричный блочныый шифр СТБ 34.101.31 «БелТ» и классические режимы работы ECB, OFB с поддержкой алгоритма дополнения блоков (Padding).